

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 3**

**COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN**  
**COURSE CODE: CSA11**

---

### **COURSE OUTCOME COVERED**

***CO4:***

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

*Assessment Tool Weightage: 30%*

---

**QUESTIONS: 6**

**Total Marks:30**

Annexure A

### **Reverse Engineering Task**

---

***Code of Conduct:***

*I M Nithishkumaran 192320037(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

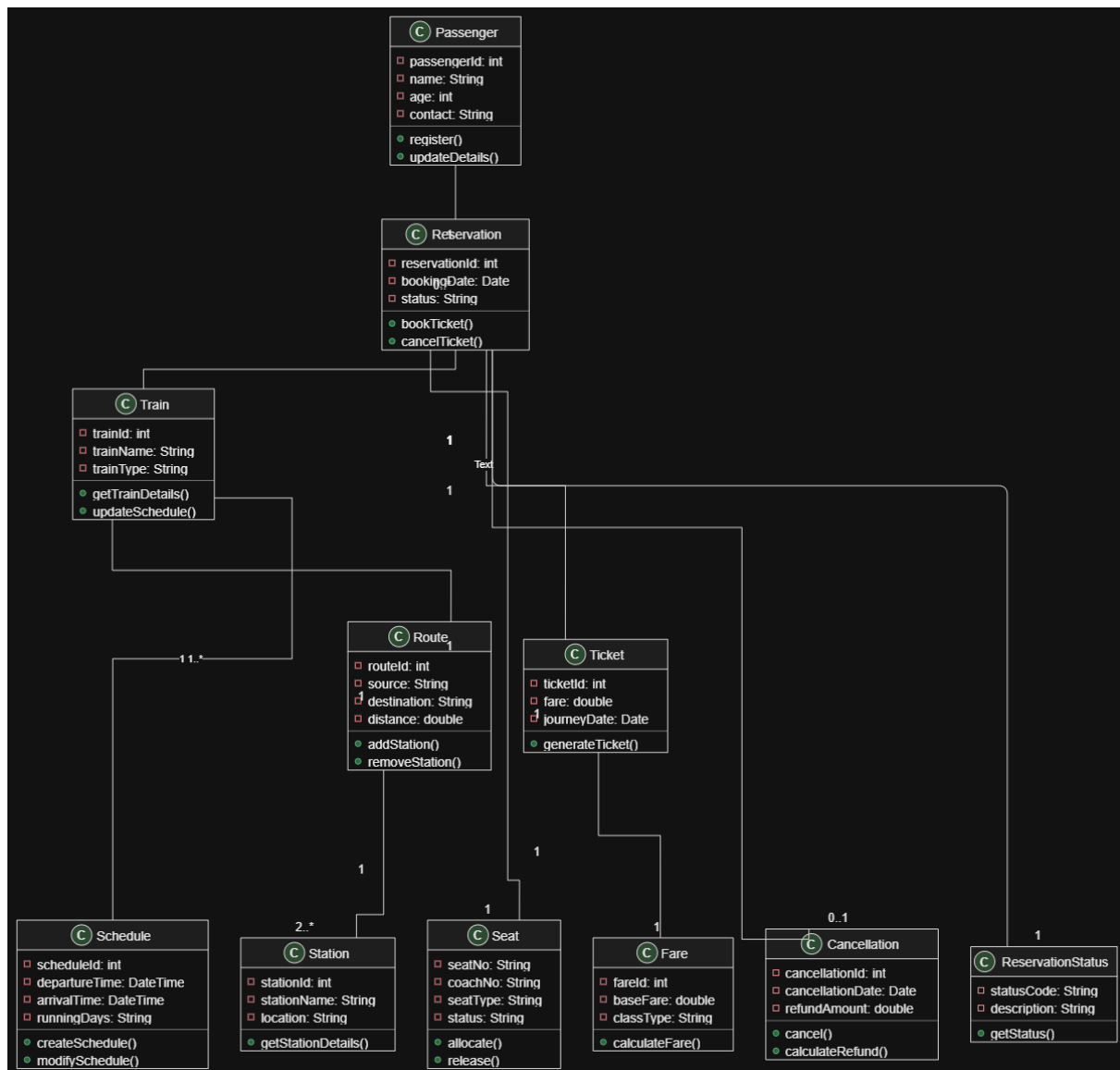
**Nithishkumaran**

## Annexure A

### Reverse Engineering Task

#### 1. Railway Reservation and Scheduling System

A legacy Railway Reservation System manages train schedules, route information, passenger registration, seat allocation, ticket booking, cancellation, fare calculation, and reservation status through a tightly coupled application. Changes in scheduling or fare rules require modifications across several modules. **Reverse engineer the existing system by identifying candidate classes, attributes, operations, responsibilities, and relationships. Develop a UML class diagram and redesign the system using object-oriented principles and layered architecture. Explain how the redesigned architecture improves cohesion, reduces coupling, and supports scalability and maintainability.**



## **Redesigned Architecture and Its Benefits**

The legacy Railway Reservation System is redesigned using object-oriented principles and layered architecture. The system is divided into separate layers such as the Presentation Layer, Service/Business Layer, Domain Layer, and Data Access Layer. The domain layer contains classes such as Passenger, Reservation, Train, Schedule, Route, Station, Seat, Ticket, Fare, Cancellation, and ReservationStatus.

### **1. Improved Cohesion**

The redesigned system improves cohesion by assigning each class a specific responsibility.

- Passenger manages passenger information and registration.
- Train and Schedule manage train and scheduling information.
- Reservation manages booking and cancellation operations.
- Seat manages seat allocation and release.
- Fare is responsible for fare calculation.
- Ticket manages ticket generation.
- Cancellation handles cancellation and refund processing.

Therefore, related data and operations are kept together within the appropriate classes. This makes each class focused, easier to understand, test, and maintain.

### **2. Reduced Coupling**

The layered architecture reduces coupling by separating different parts of the system.

The Presentation Layer communicates with the Service Layer, while the Service Layer interacts with the Domain and Data Access Layers. The user interface does not directly access the database or modify business rules.

For example, if the fare calculation rules change, modifications can mainly be made to the Fare class or its associated service instead of changing passenger registration, ticketing, or scheduling modules.

Thus, changes in one part of the system have less impact on other parts.

### **3. Scalability**

The redesigned architecture supports scalability because individual modules can be extended independently.

For example, new features such as:

- online payment,
- waitlist management,
- dynamic pricing,
- seat preference,
- mobile ticketing,

can be added as new classes or services without completely modifying the existing system.

The layered structure also makes it easier to replace or upgrade components, such as changing the database or adding a new user interface.

#### **4. Maintainability**

The system becomes easier to maintain because responsibilities are clearly separated.

Developers can locate and modify the required class or layer without examining the entire application.

For example, changes to train schedules can be handled by the Schedule and Train components, while changes to refund rules can be handled by Cancellation. This reduces the possibility of introducing errors into unrelated parts of the system.

#### **Conclusion**

The redesigned object-oriented and layered architecture transforms the tightly coupled legacy system into a modular and well-structured system. High cohesion keeps related responsibilities together, while low coupling reduces dependencies between modules. As a result, the Railway Reservation System becomes easier to extend, test, maintain, and scale.

## RUBRICS FOR CALCULATION

### Reverse Engineering Task

| Criteria                     | Weightage | Exemplary (4)                                                                             | Proficient (3)                                     | Developing (2)                                   | Beginning (1)                              |
|------------------------------|-----------|-------------------------------------------------------------------------------------------|----------------------------------------------------|--------------------------------------------------|--------------------------------------------|
| Understanding of System      | 20%       | Demonstrates clear and in-depth understanding of the system/product and its functionality | Shows good understanding with minor gaps           | Partial understanding; misses key aspects        | Lacks understanding of the system/product  |
| Decomposition                | 25%       | Effectively breaks down the system into components with detailed analysis                 | Decomposes system with minor gaps in analysis      | Limited or incomplete decomposition              | Unable to analyze or break down system     |
| Identification of Components | 20%       | Accurately identifies all components and explains their functions clearly                 | Identifies most components with minor errors       | Identifies few components; explanations are weak | Unable to identify components or functions |
| Reconstruction               | 20%       | Successfully reconstructs or models the system with accuracy and logic                    | Reconstruction is mostly correct with minor issues | Partial reconstruction; lacks clarity            | Unable to reconstruct or model system      |
| Documentation                | 15%       | Well-organized, clear documentation with diagrams and structured explanation              | Documentation is adequate with minor issues        | Poorly organized or incomplete documentation     | No proper documentation or presentation    |